Software Engineering Lab

Lab 22

N+1 Query Problem

Membunuh Latensi Tersembunyi dengan Eager Loading



Lukman / lukman-ss

Masalah Nyata: N+1 Query Menyembunyikan Latensi

Kode berfungsi sempurna pada data kecil, tetapi anjlok di produksi.

SILENT LATENCY

Poin 1

Setiap query individual berlatensi rendah (sub-milidetik).

SILENT LATENCY

Poin 2

Latensi akumulatif tumbuh linear dengan jumlah record.

SILENT LATENCY

Poin 3

Pada dataset besar, request bisa memakan waktu berdetik-detik.

CONNECTION POOL EXHAUSTION

Poin 1

Setiap request N+1 menghabiskan banyak slot koneksi database.

CONNECTION POOL EXHAUSTION

Poin 2

Under concurrent load, pool terdrain secara sequential.

CONNECTION POOL EXHAUSTION

Poin 3

Bisa menyebabkan connection timeout cascade.

HARD TO DETECT

Poin 1

Slow query logs tidak menangkap karena tiap query cepat.

HARD TO DETECT

Poin 2

Performa hanya terlihat jelas pada skala produksi.

HARD TO DETECT

Poin 3

Unit test tanpa query counter gagal mendeteksi masalah.

Masalah N+1 query sering lolos dari deteksi selama pengembangan karena dataset uji coba berukuran kecil. Setiap query individual berjalan cepat, sehingga developer tidak menyadari bahwa pola loop dengan query di dalamnya akan menghasilkan N kali round-trip ke database. Pada produksi dengan ribuan record, latensi akumulatif ini dapat meningkatkan response time dari ratusan milidetik menjadi beberapa detik. Selain itu, pola ini juga menekan connection pool database karena setiap request memerlukan banyak koneksi sequential.

Small dataset testing hides N+1; production scale exposes accumulated query overhead.

Mengapa Pendekatan Naif Gagal?

Kode N+1 terlihat sederhana tetapi tidak scalable.

```
func GetAuthorsWithPosts() []AuthorWithPosts {
    authors := db.GetAllAuthors()
    var result []AuthorWithPosts
    for _, author := range authors {
        posts := db.GetPostsByAuthorID(author.ID)
        result = append(result, AuthorWithPosts{...})
    }
    return result
}
```

Implementasi naive N+1 query tampaknya intuitif dan mudah dibaca: ambil semua author, lalu iterasi setiap author untuk mengambil posts-nya satu per satu. Namun, pendekatan ini menghasilkan jumlah query yang linear terhadap jumlah record parent. Pada sistem dengan 1000 author, akan terjadi 1001 query tunggal. Setiap query memerlukan handshake TCP, parsing SQL, eksekusi, dan pengiriman hasil. Akumulasi overhead ini menyebabkan latensi yang dapat ditoleransi pada dataset kecil tetapi menjadi bottlenecks serius pada skala produksi.

FATALITAS

Query count = 1 + len(authors).
Database connection dipakai berulang-ulang.
Tidak ada batching atau prefetching.

Naive nested loops produce O(N) queries; batching collapses it to O(1).

Mental Model: Eager Loading via Batching

Kumpulkan semua ID relasi, jalankan satu query batch, kelompokkan di memori.

```
type Author struct { ID int; Name string }
type Post struct { ID int; AuthorID int; Title string }
type AuthorWithPosts struct {
    Author Author
    Posts  []Post
}
```

PRINSIP INTI

Batch all related IDs: [author1, author2, ...]
Single query: SELECT * FROM posts WHERE author_id IN (...)
Group results in memory by author_id
Compose final structs without additional queries

Mental model inti untuk menyelesaikan N+1 query problem adalah eager loading melalui batching. Alih-alih melakukan query terpisah untuk setiap entitas parent, kita mengumpulkan semua identifier relasi yang dibutuhkan dalam satu slice. Kemudian kita menjalankan satu query batch dengan klausa IN yang mengambil semua record child sekaligus. Setelah hasil dikembalikan, kita mengelompokkannya di memori menggunakan map, lalu menyusun struktur data akhir tanpa query tambahan. Pendekatan ini mengubah kompleksitas query dari O(N) menjadi O(1).

Collect IDs, run one batch query, group in memory → replace N queries with 2.

Alur Eksekusi: Dari Loop Naif ke Batch Eager

Komparasi pipeline N+1 versus eager loading batching.

N+1 PIPELINE

1. App → DB: GetAllAuthors()
DB → App: [Alice, Bob, Charlie]
2. Loop: for author in authors
App → DB: GetPostsByAuthorID(author.ID)
DB → App: posts untuk author
Total: 4 queries (1 + 3)

EAGER PIPELINE

1. App → DB: GetAllAuthors()
DB → App: [Alice, Bob, Charlie]
2. Extract IDs: [1, 2, 3]
3. App → DB: GetPostsByAuthorIDs([1,2,3])
DB → App: ALL posts
4. Group by AuthorID in map
5. Compose AuthorWithPosts
Total: 2 queries (1 + 1 batch)

Pipeline eksekusi N+1 dimulai dengan satu query untuk mengambil semua author, kemudian masuk ke loop untuk mengambil posts setiap author secara individual. Setiap iterasi loop menghasilkan satu round-trip database tambahan. Sebaliknya, pipeline eager loading juga memulai dengan query authors yang sama, tetapi kemudian mengekstrak semua author ID dan melakukan satu query batch menggunakan klausa IN. Hasil batch dikelompokkan dalam map memori sebelum akhirnya disusun ke struktur AuthorWithPosts. Perbedaan mendasar terletak pada pengurangan round-trip dari N menjadi 1.

Batching consolidates N sequential queries into one parallel fetch.

Implementasi Eager Loading Batching

Repository.go: GetAuthorsWithPostsEager — batch query + in-memory grouping.

```
func (r *Repository) GetAuthorsWithPostsEager() []AuthorWithPosts {
    authors := r.store.GetAllAuthors()
    if len(authors) == 0 {
        return []AuthorWithPosts{}
    }

    authorIDs := make([]int, len(authors))
    for i, author := range authors {
        authorIDs[i] = author.ID
    }

    allPosts := r.store.GetPostsByAuthorIDs(authorIDs)

    postsByAuthor := make(map[int][]Post)
    for _, post := range allPosts {
        postsByAuthor[post.AuthorID] = append(postsByAuthor[post.AuthorID], post)
    }

    var result []AuthorWithPosts
    for _, author := range authors {
        result = append(result, AuthorWithPosts{
            Author: author,
            Posts:  postsByAuthor[author.ID],
        })
    }
```

Extract IDs → batch query → group in map → compose result = 2 queries total.

Implementasi eager loading batching pada method GetAuthorsWithPostsEager terdiri dari lima tahap utama. Pertama, mengambil semua author dari store. Kedua, mengekstrak semua author ID ke dalam slice integer. Ketiga, menjalankan satu query batch GetPostsByAuthorIDs dengan slice ID tersebut. Keempat, mengelompokkan semua post yang dikembalikan ke dalam map berdasarkan author_id. Kelima, menyusun struktur AuthorWithPosts akhir dengan iterasi author dan mengambil slice post dari map. Total query yang dieksekusi hanya dua: satu untuk authors dan satu untuk semua posts.

internal/blog/repository.go:32–58

Failure Path: N+1 Menghasilkan Query Count Tinggi

Bukti dari TestGetAuthorsWithPostsNPlusOne: 4 queries untuk 3 authors.

```
func TestGetAuthorsWithPostsNPlusOne(t *testing.T) {
    store := NewStore()
    repo := NewRepository(store)
    store.ResetQueryCount()
    result := repo.GetAuthorsWithPostsNPlusOne()
    queryCount := store.GetQueryCount()
    if queryCount != 4 {
        t.Errorf("expected 4 queries (N+1), got %d", queryCount)
    }
}
```

Failure path N+1 query problem termanifestasi sebagai query count yang tumbuh linear seiring bertambahnya jumlah record parent. Test GetAuthorsWithPostsNPlusOne secara eksplisit mengassert bahwa untuk 3 author, total query yang dieksekusi adalah 4. Angka ini berasal dari satu query GetAllAuthors ditambah tiga query GetPostsByAuthorID dalam loop. Pada skala produksi dengan ribuan author, pola ini akan menghasilkan ribuan query tambahan yang secara signifikan membebani database dan connection pool.

ANALISIS

Query count = 1 (getAllAuthors) + 3 (per-author loop).
Setiap iterasi loop memanggil GetPostsByAuthorID().
Pola ini akan menjadi 1+N untuk N author.

Linear query growth (1+N) is the fingerprint of N+1 anti-pattern.

Mekanisme vs Gejala: N+1 vs Eager Loading

Tabel perbandingan perilaku kuantitatif kedua pendekatan.

Aspek	N+1 (Naive)	Eager Loading (Batch)
Query Count	1 + N	2
Time Complexity	O(N) queries	O(1) queries
Connection Usage	N sequential connections	2 connections
Memory Overhead	Lower (streaming)	Higher (batch load)
Detectability	Hard (each query fast)	Easy (query counter)

Tabel perbandingan ini menyoroti perbedaan fundamental antara pendekatan N+1 naive dan eager loading batching. Dari segi kuantitas query, N+1 menghasilkan 1+N query sedangkan eager loading hanya 2 query terlepas dari ukuran dataset. Kompleksitas waktu dalam hal jumlah query juga berbeda: O(N) versus O(1). Penggunaan koneksi database pun berbeda secara signifikan; N+1 membutuhkan N koneksi sequential sedangkan eager loading hanya 2 koneksi. Namun, eager loading memiliki overhead memori yang lebih tinggi karena memuat semua record sekaligus ke dalam memori.

Trade-off: fewer queries at the cost of higher memory consumption.

3 Keputusan Desain Kritis untuk Optimasi Query

Urutan keputusan arsitektural yang harus diambil saat mengatasi N+1.

KEPUTUSAN 1: QUERY COUNTER

Implementasikan query counter pada data access layer untuk mengukur jumlah query aktual.
Gunakan counter ini dalam unit test untuk mencegah regressi N+1.

KEPUTUSAN 2: BATCH STRATEGY

Pilih strategi batching: single batch untuk dataset kecil, chunked batch untuk dataset besar.
Gunakan klausa IN dengan limit ukuran batch untuk menghindari cartesian explosion.

KEPUTUSAN 3: MEMORY VS QUERY TRADE-OFF

Evaluasi trade-off antara jumlah query dan penggunaan memori.
Untuk relasi one-to-many besar, pertimbangkan column selection atau pagination.

Tiga keputusan desain kritis ini membentuk fondasi implementasi optimasi N+1 query yang robust. Keputusan pertama menetapkan mechanisme pengukuran kuantitatif melalui query counter yang terintegrasi dengan test suite. Keputusan kedua memilih strategi batching yang sesuai dengan karakteristik dataset dan batasan memori. Keputusan ketiga mengevaluasi trade-off antara pengurangan query dan peningkatan penggunaan memori, yang menjadi pertimbangan penting saat menangani relasi one-to-many dalam skala besar.

Measure → Batch → Trade-off: the optimization decision triad.

Bukti Eksekusi: Terminal Output Demo

Output langsung dari go run ./cmd/demo yang menunjukkan perbedaan query count.

DEMO TERMINAL

```
=== 1. Simulating N+1 Query Problem ---
Loaded 3 authors with their posts.
Total queries executed: 4 (1 query for authors + 3 queries for posts)

--- 2. Simulating Eager Loading (Batching) ---
Loaded 3 authors with their posts.
Total queries executed: 2 (1 query for authors + 1 batched query for posts)
```

Terminal output dari demo CLI secara eksplisit menunjukkan perbedaan kuantitatif antara pendekatan N+1 dan eager loading. Untuk 3 author, pendekatan N+1 menghasilkan 4 query total, sedangkan eager loading hanya 2 query. Perbedaan ini terlihat jelas meskipun pada dataset kecil; pada skala produksi dengan ratusan atau ribuan record, gap antara 1+N query versus 2 query akan menjadi sangat signifikan. Demo ini membuktikan bahwa batching dapat mengurangi jumlah database round-trip hingga separuhnya.

Demo output provides concrete evidence of query count reduction from N+1 to 2.

Analisis Batasan: Yang Tidak Dibuktikan Lab Ini

Heading kanan: YANG TIDAK DIBUKTIKAN — batasan tegas dari demonstrasi.

YANG DIBUKTIKAN

Query count reduction dari 4 ke 2 untuk 3 author.
Deep equality antara hasil N+1 dan eager loading.
Thread-safety dengan race detector.
Edge case empty store.

YANG TIDAK DIBUKTIKAN

Cartesian explosion pada relasi one-to-many besar.
Memory bloat akibat eager loading massive dataset.
Performa pada database relational sesungguhnya.
Trade-off dengan join-based eager loading.
Column selection dan aggregation optimization.

Laboratorium ini memiliki batasan-batasan tegas yang perlu dipahami agar klaim tidak berlebihan. Yang dibuktikan adalah reduksi query count pada dataset kecil, kesamaan semantik hasil, thread-safety, dan penanganan edge case kosong. Yang tidak dibuktikan meliputi cartesian explosion pada relasi one-to-many berskala besar, memory bloat akibat memuat dataset massive secara eager, performa aktual pada database relational sesungguhnya, trade-off dengan strategi eager loading berbasis join, serta optimasi column selection dan aggregation.

Proof covers query count; unproven: scale, memory, join trade-offs.

Trade-off Arsitektur: Memory vs Query Count

Eager loading mengurangi query tetapi meningkatkan memori dan potensi cartesian explosion.

PROS EAGER LOADING

Query count konstan (O(1)) terlepas dari ukuran dataset.
Mengurangi connection pool pressure.
Menghilangkan latency akumulatif dari banyak round-trip.

CONS EAGER LOADING

Memory usage meningkat karena memuat semua record sekaligus.
Potensi cartesian explosion pada relasi one-to-many/many-to-many.
Perlu pagination atau chunking untuk dataset besar.
Kompleksitas kode lebih tinggi daripada N+1 naive.

Trade-off arsitektural antara N+1 naive dan eager loading batching melibatkan pertukaran antara penggunaan query dan konsumsi memori. Eager loading secara signifikan mengurangi jumlah query database dari linear menjadi konstan, yang menghasilkan penghematan latency jaringan dan pengurangan tekanan pada connection pool. Namun, pengorbanannya adalah peningkatan penggunaan memori karena semua record dimuat sekaligus ke dalam memori aplikasi. Pada relasi one-to-many berskala besar, ini dapat menyebabkan cartesian product explosion jika tidak ditangani dengan chunking atau pagination yang tepat.

Fewer queries at the cost of higher memory; scale dictates the optimal strategy.

Concurrency Safety: Thread-Safe Query Counter

Store.go menggunakan sync.Mutex untuk melindungi queryCount dan data.

```
type Store struct {
    mu           sync.Mutex
    authors      []Author
    posts        []Post
    queryCount   int
}

func (s *Store) GetQueryCount() int {
    s.mu.Lock()
    defer s.mu.Unlock()
    return s.queryCount
}
```

Implementasi store menggunakan sync.Mutex untuk memastikan thread-safety saat mengakses dan memodifikasi queryCount serta data koleksi. Setiap metode retrieval seperti GetAllAuthors, GetPostsByAuthorID, dan GetPostsByAuthorIDs mengunci mutex sebelum increment queryCount dan membebek hasilnya. Race detector Go mengonfirmasi bahwa tidak ada data race pada implementasi ini. Sifat thread-safe ini penting karena unit test dapat dijalankan secara paralel dan demo dapat diakses dari multiple goroutine.

RACE DETECTOR

$ go test -race ./...
PASS
ok github.com/.../internal/blog (cached)

Mutex-protected state enables safe concurrent testing and production use.

internal/blog/store.go:1-80

Before vs After: N+1 vs Eager Loading

Perbandingan visual antara dua pendekatan pada dataset identik.

BEFORE: N+1 (NAIVE)

Query Count: 4
Time Complexity: O(N)
Code Simplicity: High
Scalability: Poor
Memory Usage: Low

AFTER: EAGER LOADING

Query Count: 2
Time Complexity: O(1)
Code Simplicity: Medium
Scalability: Good
Memory Usage: Higher

Perbandingan before dan after menunjukkan transformasi dari pendekatan N+1 naive ke eager loading batching. Sebelum optimasi, query count sebesar 4 dengan kompleksitas waktu O(N) yang berarti skalabilitas buruk pada dataset besar. Setelah optimasi, query count turun menjadi 2 dengan kompleksitas waktu O(1) yang menjamin performa konsisten terlepas dari ukuran dataset. Pertukaran utamanya adalah peningkatan memory usage dan sedikit penurunan code simplicity karena perlu logic batching dan grouping.

Before: 4 queries O(N); After: 2 queries O(1). Scale dictates the winner.

Analisis Mendalam: TestGetAuthorsWithPostsEager

Verifikasi query count, deep equality, dan edge case kosong.

```
func TestGetAuthorsWithPostsEager(t *testing.T) {
    store := NewStore()
    repo := NewRepository(store)
    store.ResetQueryCount()
    result := repo.GetAuthorsWithPostsEager()
    queryCount := store.GetQueryCount()
    if queryCount != 2 {
        t.Errorf("expected 2 queries (eager loading), got %d", queryCount)
    }
    nPlusOneResult := repo.GetAuthorsWithPostsNPlusOne()
    if !reflect.DeepEqual(result, nPlusOneResult) {
        t.Errorf("eager result mismatch with N+1 result")
    }
}
```

Test GetAuthorsWithPostsEager melakukan tiga verifikasi sekaligus. Pertama, memastikan query count tepat 2 setelah eksekusi eager loading. Kedua, membandingkan hasil eager loading dengan hasil N+1 menggunakan reflect.DeepEqual untuk menjamin kesamaan semantik. Ketiga, implicitly menguji bahwa hasil tidak berubah meskipun mekanisme fetching berbeda. Kombinasi verifikasi kuantitatif dan kualitatif ini memberikan confidence tinggi bahwa optimasi tidak mengubah perilaku bisnis.

Query count assertion + deep equality = correctness proof for batching.

internal/blog/repository_test.go:27–50

Analisis Mendalam: TestEmptyStore

Edge case kosong diuji untuk kedua pendekatan simultan.

```
func TestEmptyStore(t *testing.T) {
    store := &Store{authors: nil, posts: nil}
    repo := NewRepository(store)
    n1 := repo.GetAuthorsWithPostsNPlusOne()
    eager := repo.GetAuthorsWithPostsEager()
    if len(n1) != 0 || len(eager) != 0 {
        t.Fatalf("expected empty results")
    }
    if !reflect.DeepEqual(n1, eager) {
        t.Errorf("empty result mismatch")
    }
}
```

Test EmptyStore menguji edge case ketika store tidak memiliki author maupun post. Kedua pendekatan, N+1 dan eager loading, harus mengembalikan slice kosong dengan panjang nol. Selain itu, deep equality antara hasil keduanya juga diverifikasi. Test ini memastikan bahwa optimasi batching tidak memperkenalkan bug pada kasus ekstrem di mana koleksi sumber kosong. Empty check pada awal method GetAuthorsWithPostsEager mencegahpanic akibat iterasi slice kosong.

Empty collection edge case must yield identical nil/empty results.

Analisis Mendalam: Race Detector Verification

Go race detector memastikan thread-safety tanpa data race.

RACE DETECTION

```
$ go test -race ./...
=== RUN TestGetAuthorsWithPostsNPlusOne
--- PASS: TestGetAuthorsWithPostsNPlusOne (0.00s)
=== RUN TestGetAuthorsWithPostsEager
--- PASS: TestGetAuthorsWithPostsEager (0.00s)
=== RUN TestEmptyStore
--- PASS: TestEmptyStore (0.00s)
PASS
ok github.com/lukman/.../internal/blog
```

Go race detector dijalankan untuk memverifikasi bahwa tidak ada data race pada akses concurrent ke store. Hasilnya menunjukkan semua test pass tanpa warning apapun. Thread-safety dijamin oleh penggunaan sync.Mutex pada setiap metode store yang mengakses atau memodifikasi queryCount dan slice data. Sifat ini memungkinkan test dijalankan secara paralel dan metode repository dapat dipanggil dari multiple goroutine tanpa risiko concurrent map write atau slice corruption.

Mutex protection + race detector = production-ready concurrency.

Analisis Mendalam: Query Counter sebagai Test Double

Store.go mensimulasikan database dengan query counter untuk verifikasi kuantitatif.

```
type Store struct {
    mu         sync.Mutex
    queryCount int
}

func (s *Store) GetAllAuthors() []Author {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.queryCount++
    return s.authors
}

func (s *Store) GetQueryCount() int {
    s.mu.Lock()
    defer s.mu.Unlock()
    return s.queryCount
}
```

Query counter pattern pada Store.go berfungsi sebagai test double yang mensimulasikan database dengan kemampuan pelacakan query. Setiap metode retrieval meningkatkan queryCount setelah mengambil lock. Method GetQueryCount memungkinkan test mengakses jumlah query aktual tanpa mengganggu state internal. Pola ini memungkinkan verifikasi kuantitatif atas optimasi query tanpa perlu instrumentasi database sesungguhnya. Implementasi ini juga thread-safe berkat penggunaan mutex.

Test double with query counter provides measurable proof of optimization.

internal/blog/store.go:1–40

Analisis Mendalam: Deep Equality Assurance

Reflect.DeepEqual memastikan hasil eager loading identik dengan N+1.

```
if !reflect.DeepEqual(result, nPlusOneResult) {
    t.Errorf("eager result mismatch with N+1 result:\nEager: %+v\nN+1: %+v", result, nPlusOneResult)
}
```

MAKNA ASSERTION

Structural equality: semua field Author dan Posts harus sama.
Order preservation: slice order harus identik.
Nil safety: empty slice vs nil slice dianggap berbeda.

Deep equality check menggunakan reflect.DeepEqual memastikan bahwa hasil eager loading secara struktural identik dengan hasil N+1. Assertion ini memverifikasi bahwa semua field pada struct Author dan slice Posts memiliki nilai yang sama. Order preservation juga dijamin karena kedua pendekatan mengiterasi author dalam urutan yang sama. Hal ini penting untuk membuktikan bahwa optimasi hanya mengubah mekanisme fetching, bukan mengubah semantik hasil.

Deep equality = behavioral equivalence between naive and optimized approaches.

Checklist Implementasi Eager Loading

Daftar kontrol sebelum deploy ke production.

- ✓ [ ] Profiling query count dilakukan sebelum deploy ke production
- ✓ [ ] Lazy loading dalam loop diaudit dan diganti dengan batch/eager loading
- ✓ [ ] Eager loading dilakukan per-query, bukan per-mapping (hindari FetchType.EAGER global)
- ✓ [ ] Cartesian explosion dicek untuk relasi one-to-many/many-to-many
- ✓ [ ] Column selection (SELECT col1, col2) digunakan ketika full entity tidak dibutuhkan
- ✓ [ ] Pagination diterapkan untuk hasil yang besar
- ✓ [ ] Query count diuji otomatis (unit/integration test)
- ✓ [ ] Strict loading mode dipertimbangkan untuk ORM yang mendukungnya

Checklist ensures systematic adoption of eager loading best practices.

Anti-Pattern: Mapping-Level Eager Loading

FetchType.EAGER global menyebabkan memory bloat, berbeda dengan query-level batching.

```
@Entity
public class Author {
    @Id
    private Long id;

    @OneToMany(fetch = FetchType.EAGER)  // ANTI-PATTERN
    private List posts;
}
```

Mapping-level eager loading seperti FetchType.EAGER pada JPA adalah anti-pattern yang sering disalahpahami sebagai solusi N+1. Meskipun memang mengurangi jumlah query, pendekatan ini memuat semua relasi yang dideklarasikan secara global tanpa seleksi. Akibatnya, entitas yang tidak relevan juga ikut ter-load ke memori, menyebabkan memory bloat dan potensi out-of-memory error pada dataset besar. Lab ini mendemonstrasikan query-level eager loading yang selektif dan dapat dikontrol per-request.

DAMPAK

Semua relasi dideklarasikan eager secara global.
Entitas tidak relevan ikut ter-load.
Memory bloat dan kemungkinan OOM pada dataset besar.
Berbeda dengan query-level eager loading yang selektif.

Global eager fetching causes memory bloat; prefer query-level selective batching.

Takeaway Praktis: Cara Mencegah N+1

Command block untuk menjalankan demo dan test validasi.

COMMAND BLOCK

```
# Jalankan demo N+1 vs Eager Loading
go run ./cmd/demo

# Jalankan test dengan query counter
go test -v ./internal/blog

# Race detection
go test -race ./...

# Profile query count
go test -count=10 ./internal/blog
```

Takeaway praktis untuk mencegah N+1 query problem adalah menerapkan profiling query count sebagai bagian dari workflow development. Command demo menunjukkan perbandingan kuantitatif antara pendekatan naive dan eager loading. Command test dengan flag verbose menampilkan detail eksekusi setiap test case. Race detection memastikan thread-safety, sedangkan repeated test counting membantu mengidentifikasi flaky test. Integrasi query counter ke dalam CI/CD pipeline akan mencegah regressi N+1 secara otomatis.

Profile → Verify → Integrate: make query counting part of your CI.

Prinsip Arsitektural Jangka Panjang

Eager loading batching sebagai pola fundamental dalam data access layer.

PRINSIP UTAMA

Minimize database round-trips: batching > sequential queries.
Measure before optimizing: query counter provides concrete evidence.
Trade memory for latency: accept higher memory usage for lower query count.
Prefer query-level over mapping-level: selective vs global eager loading.
Integrate into CI: automated query count tests prevent regressions.

Prinsip arsitektural jangka panjang untuk mengatasi N+1 query problem adalah mengadopsi batching sebagai pola fundamental dalam data access layer. Minimalisasi database round-trip melalui batching memberikan dampak performa yang lebih besar dibandingkan optimasi query individual. Pengukuran sebelum optimasi menggunakan query counter memberikan bukti kuantitatif yang dapat dipercaya. Pertukaran memory untuk latency adalah trade-off yang umumnya menguntungkan dalam sistem web modern.

Batching is fundamental; measure, trade memory for latency, integrate into CI.

Terima Kasih

N+1 solved through measurement, batching, and automated verification.

Software Engineering Lab 22 – N+1 Query Problem

Repo: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/22-n-plus-one-query-problem
Commit: 13224b48a35ba08f4a350b7d160ae4826f9e1f55
Author: Lukman / lukman-ss

Lab seri Software Engineering Lab
Oleh: Lukman / lukman-ss

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/22-n-plus-one-query-problem`

Lukman / lukman-ss

Repo: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/22-n-plus-one-query-problem
Commit: 13224b48a35ba08f4a350b7d160ae4826f9e1f55
Author: Lukman / lukman-ss

Lab seri Software Engineering Lab
Oleh: Lukman / lukman-ss